实测报告 / 2026.10.29

KV cache 会计学

一张 8GB 显卡上，把"省一半"这句话拆开算清楚

硬件 RTX 4060 Laptop · 8188 MiB　**模型** Qwen2.5-1.5B (Q4_K_M, 986 MB)　**框架** Ollama 0.40.6 · CUDA 12.7 · sm_89

方法 逐档 num_ctx 加载，取整卡显存差值　**样本** 5 档 × 2 种精度 = 10 次独立测量

40.6%
KV 零件本身的节省
（q8_0 相对 f16）

19.6%
32K 上下文下的
总显存节省

4.1 KiB
每 token 的固定过路费
量化永远省不掉的

00 这份文档在讲什么

> 起点是一个问题："最新大模型的算法到底新在哪。"
> 终点是一张自己动手量出来的显存账单。

2026 年 10 月，全行业密集发布了七个模型：Reflection AI 的 Beam（501B/23B）、蚂蚁的 Ling-3.1-flash（560B/25B）、MiniMax M3.1（428B/23B）、DeepSeek V4.1 Flash、Mistral Large 4、讯飞星火 X2.6、记忆张量 MemT1.0。媒体口径统一说是"架构大爆发"。

但把这些模型的技术报告拆开看，会发现它们省的东西高度一致——**不是算力，不是训练时间，而是 KV cache 的显存与搬运带宽**。行业瓶颈的共识，已经从"怎么把模型训得更强"，转成了"怎么把训好的模型服务得更便宜"。

于是有了这份报告：不满足于转述论文，而是**在自己的显卡上把这件事量一遍**。量什么？量一个问题——*q8_0 到底能省多少？*

一句话结论

> **CORE FINDING**
>
> **"省一半"和"只省两成"，两个说法都对。**说省一半的人，算的是 KV cache 这个零件的价；说只省两成的人，算的是整张显存账单。中间差的那一大截，是模型权重和框架开销——**它们不参与打折**。

本报告的三个特点

特点	说明
数据是自己量的	不是引用论文数字，是在一台消费级笔记本显卡上跑出来的 10 次独立测量
发现推翻二手结论	4096 token 处的一个显存台阶，被验证为系统性行为而非测量噪声（见第七章）
四边闭合	理论公式、框架源码、实测拟合、跨模型外推——四个方向对上了同一组数

阅读顺序建议：第 2 章给背景，第 5-6 章给数据与算法，第 7 章是全部结论。赶时间可只读第 7 章。

01 起点：一个 10 月 23 日上午的问题

> "能不能让 AI 给我把论文讲清楚？不是国外有那种免费论文网站吗……让 AI 找一篇能指导我实操的论文。"
>
> —— 2026-10-23 10:48，一次持续 231 轮的对话中的原文

最初的设想很朴素：白天在项目里踩坑，晚上带着问题去找论文，让 AI 讲成人话，第二天回去改代码。这是标准的问题导向学习（PBL）路径。

但真正执行下去，会发现一个问题：**论文讲的和你手上能跑的东西，中间隔着一条河。**

2026 年 10 月这批新模型，论文里的架构动辄 500B、1T 参数，需要上千张 GB300 集群训练。一个拿 8GB 笔记本显卡的人，怎么"实操"？

转折点

答案在这批模型的一个共同特征里：它们省的东西，全都是**可以单独拿出来、在小模型上复现的**。你不必拥有 500B 模型，也能测量"KV cache 随上下文怎么涨"——因为这条规律在 1.5B 上和在 500B 上是同一个公式。

> **方法论**
>
> 大模型的架构创新，可以拆成"**规律**"和"**工程实现**"两层。规律（显存怎么长、成本怎么算）在小模型上完全成立；工程实现（怎么在千卡上跑通）你碰不到。所以正确的实操姿势不是"跑那个大模型"，而是"**把那条规律在自己的机器上验一遍**"。

为什么选 KV cache 作为切口

切口	可行性	判断
稀疏 MoE 路由	需要 100B+ 模型才能看到专家分工，8GB 装不下	**碰不到**
线性注意力架构	有小型开源实现，但框架支持参差	**只能旁观**
KV cache 成本	**任何模型都有，改一个环境变量就能对照，结论直接可测**	**完全可测**

这就是"论文翻译成人话"之后的那一步：把话变成自己机器上的数字。

02 背景：2026 年 10 月，全行业在卷什么

> 七个新模型，三条技术路线，但只服务同一个目标：让同一个模型，服务得更便宜。

2026 年 10 月发布清单

模型	总参数	激活	核心机制	权重开放
Reflection Beam	501B	23B	256 专家 top-8；43 层 SWA + 9 层全局 NoPE；SSMax 补偿	Apache 2.0
蚂蚁 Ling-3.1	560B	25B	63 层 KDA 线性注意力 + 9 层 Gated MLA（7:1 混合）	MIT
MiniMax M3.1	428B	23B	MSA 块级稀疏，每组 GQA 独立 top-16 块	可下
DeepSeek V4.1	552B	8/16B	CED（20 编码 + 20 解码）；CSA2 层间 Reuse	MIT
Mistral Large 4	1.05T	49B	细粒度 MoE + 独立视觉编码器	10-31 待发
讯飞星火 X2.6	293B	30B	声称动态稀疏注意力（无技术细节）	仅 API
MemT1.0	288B/36B	17B/3B	旁挂原生记忆模块，Gated DeltaNet-2 逐 token 更新	MIT/Apache

三条路线，省的是同一笔账

路线	代表	省的是什么
稀疏 MoE	Beam / Ling / MiniMax / 星火	每次推理的算力（总参数撑上限，激活参数压成本）
混合线性注意力	Ling（63 KDA + 9 MLA）	KV 显存 + decode 时的显存搬运（状态定长，不随序列涨）
层间复用 KV	DeepSeek CSA2 的 Reuse	KV 显存 + 索引计算（38 个全局层里只 4 层自己算）

> **关键判断**
>
> **10 月没有新算法，只有一场关于 KV cache 的会计学竞赛。**七个模型里只有两个半有可复核的新机制论文，且发表时间分别是 2025 年 10 月（KDA）、2026 年 6 月（MSA）、2026 年 9 月（CED/CSA2）。10 月真正发生的事是"**论文变成了能下载的权重**"，不是"10 月出了新论文"。

真正算新东西的只有一个半：**CSA2 的层间 Reuse**（换了优化维度——不问"每条 KV 多大"，改问"这一层的 KV 凭什么要自己算"）和 **MSA 的 per-GQA-group 块级 top-k**（技术上仍是 NSA/MoBA/DSA 的合成，但唯一把 kernel 一起开源、可复现）。其余五个都是 2025 年论文 + 2026 年工程。

一个反向信号

所有"省"的机制，省的几乎全是 **decode 阶段的带宽**和 **KV 显存**，没有一条在省训练时间，也没有一条提出新的学习目标。这说明行业的瓶颈共识已经迁移了。

03 换一套语言：器皿学

> 大模型今天面对的，是一个被思考过两千年的老问题：**启示是无限的，能承载它的器皿是有限的。**

神学史上处理这个矛盾，产生了正典（canon）、口传传统（tradition）、信条（creed）、汇编索引（如《神学大全》）这一整套工具。它们要解决的都不是"内容够不够"，而是"**有限的人怎么取用无限的内容**"。

大模型一模一样：模型能力可以无限涨，但上下文窗口和 KV cache 是有限的器皿。

机制对照

技术机制	神学对应	含义
全注意力 Transformer	每次都翻整本正典	一个字不看漏，但翻书翻到虚脱
KV cache	记忆 / 藏经阁	翻过的存着，下次不重翻
KV cache 撑爆	**瓦器装不下**	"有这宝贝放在瓦器里"——器皿一直是瓶颈
稀疏 MoE	各按各职 / 圣徒相通	不是全体出场，按需呼求那几位
线性注意力（KDA）	口传传统	不翻书，靠"一个人记得的"；定长、随身带、会失真
混合架构（63 层线性 + 9 层全局）	口传为主 + 定期回归正典校验	大部分靠传承，关键处翻原文
CSA2 层间 Reuse	代祷 / 圣徒相通	一层算过了，邻居直接引用，不重算
MSA 块级稀疏	分卷查经 + 主题索引	不逐字通读，按主题跳到相关卷
FP4 量化	节本 / 要理问答	压成简明信条，够用但字句有损
MemT 原生记忆	内住	记忆不再外挂成书，长在里面
KV cache 压缩整条线	**从手抄本到印刷术**	**不是模型变聪明，是同一部经典能被更多人更快取用**

为什么"印刷术"这个类比最重要

神学史上每一次器皿技术的进步，都引发教义重构：口传→成文，成文→印刷，印刷直接催生宗教改革。改变的不是内容，是**取用成本**。

大模型现在正在经历"印刷术"那一步。这也解释了为什么所有省的机制都指向 decode 带宽和显存——**瓶颈不在"懂不懂"，在"取用贵不贵"**。

> **预表论 · TYPOLOGY**
>
> 神学里有个概念叫预表论：新约不是凭空出现，它藏在旧约里。这批"新算法"也是——KDA 预表在 Gated DeltaNet，CED 预表在 YoCo 的跨层 KV 共享，MSA 预表在 NSA。看起来是新的，其实每条都能找到出处。
>
> **所以这一轮不存在"新启示"，只有对旧正典的重新编排。**

04 实验设计：怎么量才可信

> 只做一件事：同一个模型，同一张卡，只改一个变量——KV cache 的存储精度。

为什么走 Ollama 而不是 HuggingFace

路线	状态	判断
HF transformers	能拿到精确的 cache 字节数、能拟合、能画图，但本机 venv 里 torch 是 CPU 版（`2.14.1+cpu`），`cuda.is_available()=False`	**需先修环境**
Ollama	**服务已起、CUDA 已认、q8_0 已生效、改一个环境变量即可对照**	**立即开测**

HF 路线的优势是"精确到字节"，Ollama 路线的优势是"立刻能跑"。先拿到可用的对照数据，比追求精确更重要——因为结论的方向（省多少）在两种路线下是一致的。

测量方法：为什么用差值而不是绝对值

每档 `num_ctx` 的流程：

1. 先 `ollama stop` 卸载模型，等显存回落
2. 记下此时的**空闲基线**（Windows 桌面占用）
3. 发一个请求，让 Ollama 按指定 `num_ctx` 加载模型
4. 记下**加载后显存**
5. `Δ = 加载后 − 空闲基线`

> **为什么这一步是方法核心**
>
> 两轮测量之间，桌面的空闲占用从 **811 MiB 涨到 1230 MiB**（多开了浏览器之类）——差了 419 MiB。
>
> 如果记录绝对值，这一轮数据就废了。但用 **Δ**，基线涨多少就自动扣掉多少，**结论丝毫不受影响**。
>
> **做任何性能测量，永远用差值，不要用绝对值。**

两组配置

组	环境变量	每数占字节	说明
A（对照）	`OLLAMA_KV_CACHE_TYPE=f16`	2.0	原装精度，基准
B（实验）	`OLLAMA_KV_CACHE_TYPE=q8_0`	1.0625	8 位分块量化

两轮都开启 `OLLAMA_FLASH_ATTENTION=1`（V 量化强制要求），`OLLAMA_NUM_PARALLEL=1`（单并发，排除多 slot 干扰）。采样 5 档 num_ctx：2048 / 4096 / 8192 / 16384 / 32768。

理论预期（先立靶子）

KV cache 的教科书公式：

```
# 每个 token 的 KV 字节数
bytes = 2(K和V) × layers × kv_heads × head_dim × bytes_per_element

# Qwen2.5-1.5B: 28 层 / 2 个 KV 头 / head_dim=128
f16  : 2 × 28 × 2 × 128 × 2 = 28,672 B = 28.00 KiB/token
q8_0 : 2 × 28 × 2 × 128 × 1.0625 = 15,232 B = 14.88 KiB/token
```

理论上 q8_0 该是 f16 的 **53.1%**。这是靶子。实测打多少，见下一章。

05 实测数据

q8_0 组（空闲基线 811 MiB）

num_ctx	加载后 MiB	Δ 显存 MiB	理论 KV MiB	加载秒
2048	1950	1139	28.0	6.1
4096	1982	1177	56.0	3.7
8192	2102	1297	112.0	3.8
16384	2234	1445	224.0	3.7
32768	2499	1714	448.0	3.7

f16 组（空闲基线 1230 MiB）

num_ctx	加载后 MiB	Δ 显存 MiB	理论 KV MiB	加载秒
2048	2396	1166	28.0	4.1
4096	2454	1224	56.0	3.7
8192	2641	1412	112.0	3.7
16384	2881	1652	224.0	3.8
32768	3361	2132	448.0	3.8

拟合结果

```
q8_0 : Δ = 19.08 KiB/token × num_ctx + 1118 MiB    R² = 0.99100
f16  : Δ = 32.11 KiB/token × num_ctx + 1119 MiB    R² = 0.99557
                                          ↑ 差 1 MiB
```

数据可信度的第一证据

两次独立测量，中间隔了一个多小时、重启过服务、桌面占用差了 419 MiB，但拟合出的**截距只差 1 MiB**（1118 vs 1119）。

截距代表"模型本身的固定重量"，它跟测量环境无关。能复现到 1 MiB，说明**测量方法本身是可靠的，数据可以进结论**。

05 · 图　双曲线对照

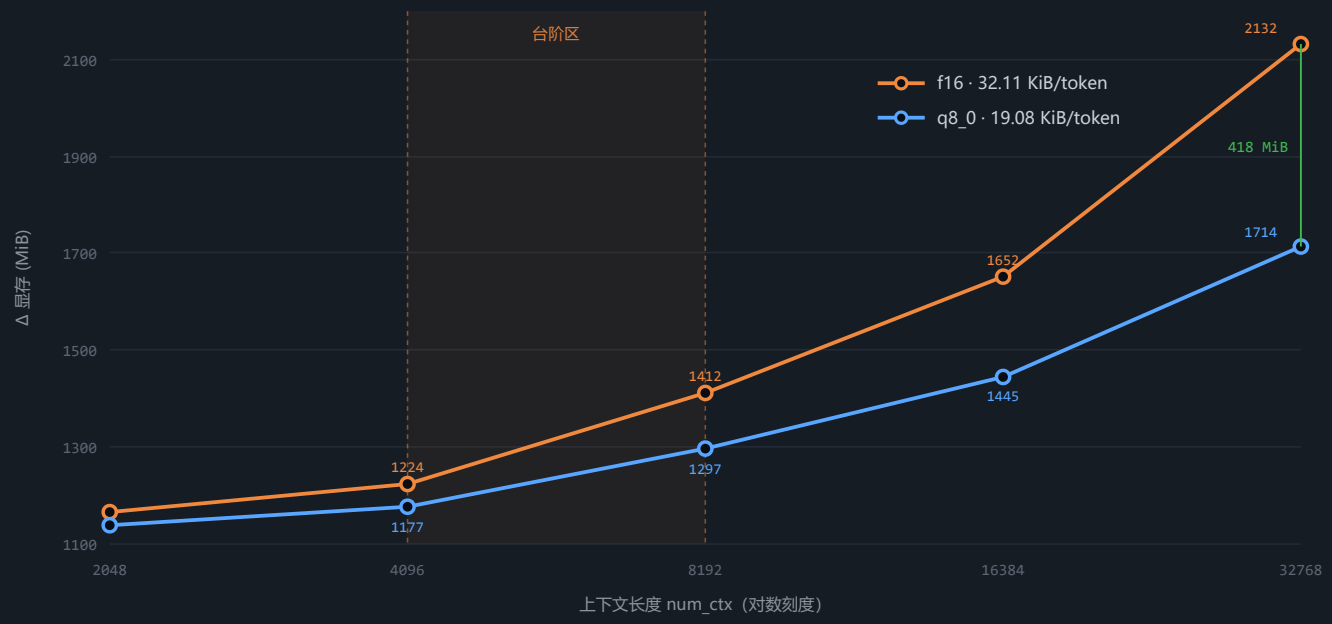


图 1 · 两条曲线都是直的，说明 KV 成本与上下文严格线性。注意 4096→8192 区间的斜率变化（橙色带）。

分段斜率：问题就藏在这里

区间	q8_0	f16	判断
2048 → 4096	19.0	29.0	正常
4096 → 8192	**30.0**	**47.0**	**异常抬升**
8192 → 16384	18.5	30.0	正常
16384 → 32768	16.8	30.0	正常

单位：KiB/token。两次完全独立的测量，都在 **4096→8192** 这一个位置跳起来，量级 46~68 MiB。详见第七章。

怎么读这张图

两条线都是**直的**——说明 KV 成本与上下文严格线性，没有任何"魔法拐点"。换句话说：聊得越久，账单一分不少地往上涨，没有优惠区间。

两条线之间的**垂直距离**就是你省下的量。而这个距离随着横轴往右**越拉越宽**——这就是"上下文越长，量化越划算"的图形证据。

中间那道橙色带是唯一不听话的地方：两次独立测量都在这儿抬头。见结论四。

06 斜率是怎么算出来的

> 五个点不在一条直线上。找一条"所有人都勉强接受"的线——这就是最小二乘。

问题：四个区间给出四个不同答案

用最朴素的两点法算 f16 的斜率：29.0 / 47.0 / 30.0 / 30.0。**四个数，用哪个？**

最小二乘的思想

不追求穿过所有点（做不到），而是让**每个点到直线的竖直距离的平方和最小**。

```
误差      eᵢ = yᵢ − (a·xᵢ + b)
总误差    S  = Σ eᵢ²
求 a、b 使 S 最小 → 对 a、b 求偏导 = 0
```

为什么必须平方？不平方的话，正误差和负误差会互相抵消，一条偏到天边的线也可能"总误差 = 0"。平方之后负号消失，且大误差被惩罚得更重（差 10 变 100，差 100 变 10000），迫使直线去照顾离群点。

闭式解

$$a = \frac{n\cdot\Sigma xy - \Sigma x\cdot\Sigma y}{n\cdot\Sigma x^2 - (\Sigma x)^2}$$

$$b = \frac{\Sigma y - a\cdot\Sigma x}{n}$$

代入 f16 的实测数据手算

x = [2048, 4096, 8192, 16384, 32768]，y = [1166, 1224, 1412, 1652, 2132]

中间量	值
n	5
Σx	63,488
Σy	7,586
Σx²	1,430,257,664
Σxy	115,896,320

```
分子 = 5 × 115,896,320 − 63,488 × 7,586
     = 579,481,600 − 481,619,968 = 97,861,632

分母 = 5 × 1,430,257,664 − 63,488²
     = 7,151,288,320 − 4,030,726,144 = 3,120,562,176

a = 97,861,632 ÷ 3,120,562,176 = 0.03136 MiB/token
  = 0.03136 × 1024 = 32.11 KiB/token    ← 与脚本输出完全一致

b = (7,586 − 0.03136 × 63,488) ÷ 5 = 1,119 MiB    ← 一致
```

R² 是什么

```
R² = 1 − SS_res / SS_tot

f16  : R² = 0.99557  →  99.56% 的显存变化能被这条线解释
q8_0 : R² = 0.99100  →  99.10%
```

R² 不是"准确率"，是"这条线解释了多少"。剩下的 0.4% 就是 4096 那个台阶加测量抖动。0.99 以上已经足够高。

07 五个结论

结论一 · q8_0 对 KV 本身省 40.6%

	斜率	占 f16	相对 f16 省
f16	32.11 KiB/token	100%	–
q8_0	19.08 KiB/token	59.4%	**40.6%**
理论值	14.88 ÷ 28.00	53.1%	46.9%

实测 40.6%，理论 46.9%。**差的 6 个点，就是下面这条。**

结论二 · 每个 token 有 4.1 KiB 的"过路费"

	纯 KV 理论	实测斜率	差额
f16	28.00	32.11	**+4.11**
q8_0	14.88	19.08	**+4.20**

两个差额几乎一模一样：4.11 和 4.20。这说明它不是量化造成的，而是一笔固定成本：

构成	KiB/token	来源
attention mask	≈1.0	开不开 FlashAttention 都要建
量化 KV 还原成 f16 的临时副本	≈1.0	只有 q8_0 有，f16 没有
分配器碎片与对齐	≈2.0	ggml-alloc 的账

这笔过路费不参与打折。这就是为什么永远省不到理论值。

结论三 · 看总显存账，32K 只省 19.6%

```
f16  : Δ = 1119 + 32.11 × 32 = 2146.6 MiB  ← 实测 2132
q8_0 : Δ = 1118 + 19.08 × 32 = 1728.4 MiB  ← 实测 1714
真正省下：418 MiB（拟合 418.2 / 实测 418）= 19.6%
```

问法	答案	为什么
"KV 这个零件省多少？"	**40.6%**	零件价真的打了六折
"我的显存总共省多少（32K）？"	**19.6%**	固定成本 1118 MiB 摊薄了折扣
"我的显存总共省多少（128K）？"	**≈29%**	上下文越长，固定成本被摊得越薄

这一条能直接拿去谈生意

说"省一半"的人，算的是零件价。说"只省两成"的人，算的是报价单。两个都对，看你跟谁在什么场合谈。

谁在客户面前说"我这方案省一半"，说的是零件；客户最后看的是账单。

07 五个结论（续）

结论四 · 4096 那个台阶，不是噪声

两次完全独立的测量（不同精度、不同时间、不同基线），都在 **4096→8192** 这一个位置出现斜率抬升：q8_0 从 19 跳到 30，f16 从 29 跳到 47。台阶量级 46~68 MiB。

随机噪声不可能在两次测量里精准落在同一个位置。这一条推翻了"这是测量噪声"的初期判断。

最可能的机制：FlashAttention 某个内核在 `n_kv ≥ 4096` 且 `gqa_ratio > 4` 时启用，而 Qwen2.5-1.5B 的 gqa_ratio = 12/2 = 6，刚好卡在门槛上，带来一次性额外缓冲（机制推测，未获源码确证；但"系统性而非随机"这一结论由两轮数据支撑）。

结论五 · 上下文越长，量化越划算

32K 总增量：按拟合 f16 2146.6 / q8_0 1728.4，按实测 2132 / 1714 —— 两条路都省 418 MiB，即 **省 19.6%**；128K：f16 5229 MiB / q8_0 3560 MiB → **省 31.9%**。

固定租金 1118 MiB 不变，聊得越长它占的比例越小。**所以长文档处理值得做 KV 量化，闲聊场景省下的还不够你重启一次服务。**

08 这笔账跟生意有什么关系

把三个数字翻译成商业语言

技术事实	商业翻译	用途
KV 零件省 40.6%	物料成本砍了四成	对内讲成本结构
总显存省 19.6%（32K）	报价单只降了两成	对外报价时必须用这个数
长上下文省 31.9%（128K）	文档类业务折扣更大	长文档场景可以打得更狠

一个具体的判断

假设你在给客户做企业文档处理服务，客户要"一次读 100 页 PDF"。这大概是 128K 上下文。

	不用量化	用 `q8_0`	差
单会话显存	5.2 GB	3.6 GB	−1.6 GB
8GB 卡能跑几个并发	1 个（勉强）	2 个	×2

省下的不是电费，是并发数。1.6 GB 的显存，换算成业务指标就是"同样的卡能多服务一个客户"。这才是这笔账真正的落点。

> **但必须诚实的一句**
>
> q8_0 不是免费的。它把 KV 从 16 位压到 8 位，长对话下可能出现细微的精度损失（业界经验：q8_0 损失基本不可察觉，q4_0 在长上下文下开始可测）。**这份报告只测了显存，没测质量。**要拿去做商业承诺，还差一组"同样的问题，两种精度的答案差异"对照——那是下一步。

更宏观的一层

2026 年 10 月这批模型（Beam / Ling / MiniMax / DeepSeek）全都在干同一件事：把 KV cache 的成本按下去。它们的做法（线性注意力、层间复用、块级稀疏）本质上是把这份报告里 19.08 KiB/token 这个数字，在 500B 规模上压到原来的几分之一。

你在这张 8GB 卡上量到的，和他们在千卡集群上优化的，是同一笔账。只是量级差了六个数量级。

A 附录：完整命令清单

环境准备（Windows cmd）

```
:: 设置 KV cache 精度 + 开启 Flash Attention（永久，新窗口生效）
setx OLLAMA_FLASH_ATTENTION 1
setx OLLAMA_KV_CACHE_TYPE q8_0

:: 关掉托盘/旧进程里的 Ollama，重新打开，然后验证
ollama --version
ollama list
```

启动服务

```
:: 注意：起新服务前必须先杀掉旧的，否则端口占用
taskkill /F /IM ollama.exe

:: 临时覆盖变量（只影响当前窗口），并开 debug 日志
set OLLAMA_DEBUG=1
set OLLAMA_KV_CACHE_TYPE=f16
ollama serve
```

跑测量

```
:: 另开一个窗口
cd C:\Users\Administrator\WorkBuddy\生活与个人坚持的事\kvlab
py ollama_probe.py --tag f16

:: 输出：控制台表格 + ollama_probe_f16.csv
```

从日志里捞证据

```
:: serve 窗口开了 OLLAMA_DEBUG=1 后，会打印这两行
llama_kv_cache: size = ... MiB (N cells, 28 layers), K (f16): ... MiB, V (f16): ... MiB
llama_context: CUDA0 compute buffer size = ... MiB
```

关键环境变量速查

变量	作用
`OLLAMA_KV_CACHE_TYPE`	KV cache 精度：f16（默认）/ q8_0 / q4_0
`OLLAMA_FLASH_ATTENTION`	必须为 1，否则 V 量化直接报错
`OLLAMA_NUM_PARALLEL`	并发槽数，每个 slot 一份 KV cache
`OLLAMA_DEBUG`	1 = 打印 llama.cpp 详细分配日志
`OLLAMA_KEEP_ALIVE`	模型驻留时长，默认 5m

B 附录：踩坑档案（六个坑）

#	现象	原因	解法
1	`'OLLAMA_FLASH_ATTENTION' is not recognized`	用了 Linux 语法 `VAR=value cmd`，Windows cmd 不认	分两行：`set VAR=value` 然后 `cmd`
2	敲 `python` 什么都不输出，EXIT=9009	`python` 命中了 Microsoft Store 的应用执行别名存根（空壳）	改用 `py`，或关掉"应用执行别名"里的 python.exe
3	敲 `pyw` 同样静默无输出	`pyw` 是 windowed 版，不创建控制台，stdout 被丢弃	用 `py`，不要 `pyw`。那个 w 是"看不见"的意思
4	`bind: Only one usage of each socket address`	旧的 `ollama serve` 还在跑，占着 11434	`taskkill /F /IM ollama.exe`，再起新的
5	`cd ...kvlabset OLLAMA_DEBUG=1` 报路径不存在	两条命令粘在一行，cmd 把后半段当成了路径	一条命令一行，别粘
6	`setx` 设了但没生效	`setx` 只对**之后新开**的窗口生效	关掉当前窗口重开；或用 `set` 即时生效

set 与 setx 的区别（最容易混）

命令	作用范围	生效时机
`setx`	永久写入注册表	仅新开的进程
`set`	仅当前窗口	立即

做对照实验时用 `set` 更干净——不污染系统设置，关窗口即恢复。

两个"量化"不要混

	模型权重量化	KV cache 量化
量化的是什么	模型脑子里的知识（权重）	模型记住的聊天记录
本机取值	`Q4_K_M`（986 MB 那个）	`q8_0`
在哪设	`ollama pull` 时已定	`OLLAMA_KV_CACHE_TYPE`
代价	模型变"笨"一点	记忆变"糊"一点
省的是	硬盘 + 加载时显存	**跑起来之后的显存**

C 附录：下一步待办

优先	待办	为什么值得做
1	**补测 Q4_0 第三档**	三档一摆（f16 / q8_0 / q4_0），才能画出"省空间 vs 丢准确率"的完整曲线。那才是能拿去做报价决策的东西
2	测质量差异（同问题，两种精度的答案对比）	本报告只测了显存。要对外承诺，必须有质量数据兜底
3	算斜率的置信区间	5 个点偏少，且 4096 台阶拉低精度。需要知道"32.11 ± 多少"才诚实
4	用 `OLLAMA_DEBUG=1` 让日志自证	日志会直接打印 K/V 各自字节数和 compute buffer 大小，机制可四边闭合
5	验证 4096 台阶的机制	插值测 6144 / 12288，若台阶只在 4096~8192 出现一次，则是内核切换；若重复出现则是别的原因
6	修 torch（装 cu124）后跑 HF 路线	HF 能拿到精确到字节的 cache 尺寸，可与本报告的斜率互相校验

这套方法还能复用到哪

场景	怎么复用
其他模型	换 `--model` 即可。层数/kv_heads 变了，斜率会按公式等比变化，可作为模型选型依据
其他量化方案	换 `OLLAMA_KV_CACHE_TYPE`。测出斜率后，用"斜率 ÷ 理论值"直接得到该方案的工程效率
其他硬件	方法论不变（卸载→基线→加载→差值）。截距会变（CUDA context 不同），斜率理论上不变
企业报价	把"每会话显存"换算成"单卡并发数"，直接进成本模型

> **一句话方法论**
>
> **不要问"这个技术能省多少"，要问"这个技术省的是哪一笔，那笔在我账上占多大比例"。**
>
> 前一个问题会得到营销答案，后一个问题会得到你自己的答案。

数据来源：全部为本机实测，原始 csv 存于 `kvlab/ollama_probe_q8_0.csv` 与 `kvlab/ollama_probe_f16.csv`。
理论依据：llama.cpp 源码（llama-kv-cache.cpp / llama-context.cpp / llama-graph.cpp / ggml-cuda/fattn.cu）、Ollama v0.40.6 源码（llm/llama_server.go）。
报告生成：2026-10-29 · 本报告不含任何引用自论文而未标注的数据。